

PROJECT: MULTI-AZ WORDPRESS DEPLOYMENT

Services: EC2, RDS (MySQL), EFS, ALB, CloudFront

Goal: Highly available WordPress deployment with content delivery optimization.

Skills: Shared storage (EFS), DNS, caching strategies.

Definition: This project demonstrates the deployment of a highly available and scalable WordPress website on AWS using a multi-AZ architecture. It integrates services such as Amazon EC2, Amazon RDS, Amazon EFS, Application Load Balancer (ALB) and CloudFront to improve reliability, performance, and content delivery. The project highlights how AWS services work together to build a resilient and production-ready web application.

Scope of the Project: This project focuses on building a highly available and scalable WordPress environment on AWS. It includes configuring EC2 instances, RDS databases, EFS shared storage, Route 53 DNS, and CloudFront content delivery. The implementation demonstrates cloud architecture best practices for load balancing, fault tolerance, and improved website performance.

Methodologies:

1. WordPress Deployment using EC2 + RDS + VPC – Method Used

- Uses AWS networking services including VPC, Subnets, Internet Gateway, Route Tables, and Security Groups.
- Deploys a WordPress application on an Amazon EC2 instance running Apache, PHP, and MySQL client.
- Uses Amazon RDS as a managed MySQL database instead of installing MySQL directly on the EC2 instance.
- Separates the application layer and database layer, improving security and scalability.
- Stores website data securely in Amazon RDS located within the VPC.
- Uses Security Groups to control communication between WordPress and the database.
- Provides high availability through deployment across multiple Availability Zones.
- Suitable for production-ready WordPress hosting environments.
- Reduces database administration effort through Amazon RDS automated backups and maintenance.
- Demonstrates cloud networking, compute, storage, and database integration within AWS.

2. WordPress Deployment using EC2 + Local MySQL – Alternative Method

- Installs both WordPress and MySQL database on the same EC2 instance.
- Requires manual installation and management of MySQL.
- Simpler to configure for testing and learning purposes.
- Lower initial setup complexity.
- Does not provide database isolation from the application server.
- Limited scalability because application and database resources share the same server.
- Requires manual database backup and maintenance.
- Less secure than using Amazon RDS.
- Suitable only for small-scale websites, testing environments, and proof-of-concept deployments.
- Not recommended for production workloads requiring scalability, reliability, and high availability.

Services Used in this Project:

1. Amazon EC2 (Elastic Compute Cloud)

- **Definition:** Amazon EC2 is a cloud computing service that provides virtual servers to run applications on AWS.
- **Purpose:** EC2 is used to host and run the WordPress website with Apache and PHP.
- **Role:** EC2 acts as the web server that handles user requests and delivers the WordPress website.

2. Amazon RDS (Relational Database Service)

- **Definition:** Amazon RDS is a managed database service that allows users to easily create, manage, and maintain relational databases in the cloud.
- **Purpose:** Amazon RDS is used to store and manage the WordPress MySQL database.
- **Role:** Amazon RDS acts as the backend database that stores website data such as posts, users, and settings.

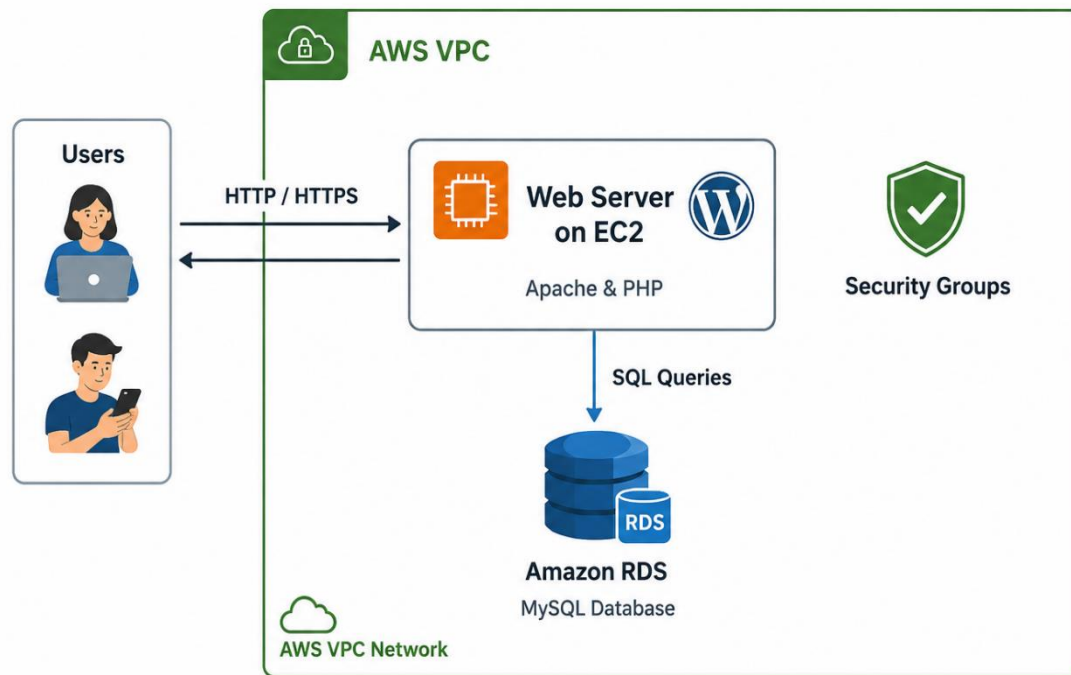
3. Security Group

- **Definition:** A Security Group is a virtual firewall in AWS that controls inbound and outbound network traffic for resources like EC2 and RDS.
- **Purpose:** The Security Group is used to allow necessary access such as HTTP, SSH, and database communication.
- **Role:** The Security Group ensures secure communication between the EC2 instance and the RDS database while protecting the server from unauthorized access.

4. VPC (Virtual Private Cloud)

- **Definition:** A VPC (Virtual Private Cloud) is a virtual network in AWS where cloud resources like EC2 and RDS are securely deployed.
- **Purpose:** The VPC provides the network environment for hosting the EC2 instance and RDS database.
- **Role:** The VPC enables secure communication between EC2 and RDS within the same network.

Architecture Diagram:



Implementation

STEP 1: Create VPC

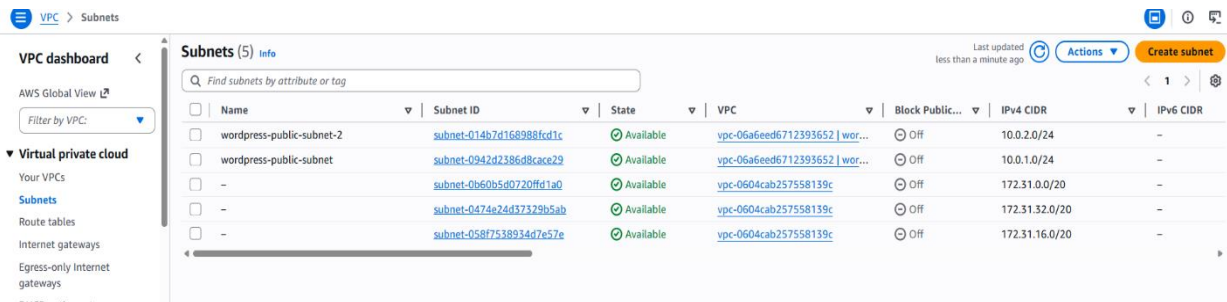
- Go to AWS Console → VPC → Create VPC.
- Resource Type: VPC Only.
- Name: wordpress-vpc.
- IPv4 CIDR Block: 10.0.0.0/16.
- Click Create VPC.
- Name: wordpress-vpc

The screenshot shows the AWS VPC console for the VPC 'vpc-06a6eed6712393652 / wordpress-vpc'. The left sidebar shows the navigation menu with 'Virtual private cloud' selected. The main content area is divided into 'Details' and 'Resource map' tabs. The 'Details' tab shows various configuration options like DNS resolution, Main network ACL, IPv6 CIDR, and Encryption control ID. The 'Resource map' tab shows a visual representation of the VPC resources, including subnets (ap-south-1a, ap-south-1b), route tables (rtb-04e4fda3b2fc020d6), and network connections (wordpress-igw).

STEP 2: Create Subnets

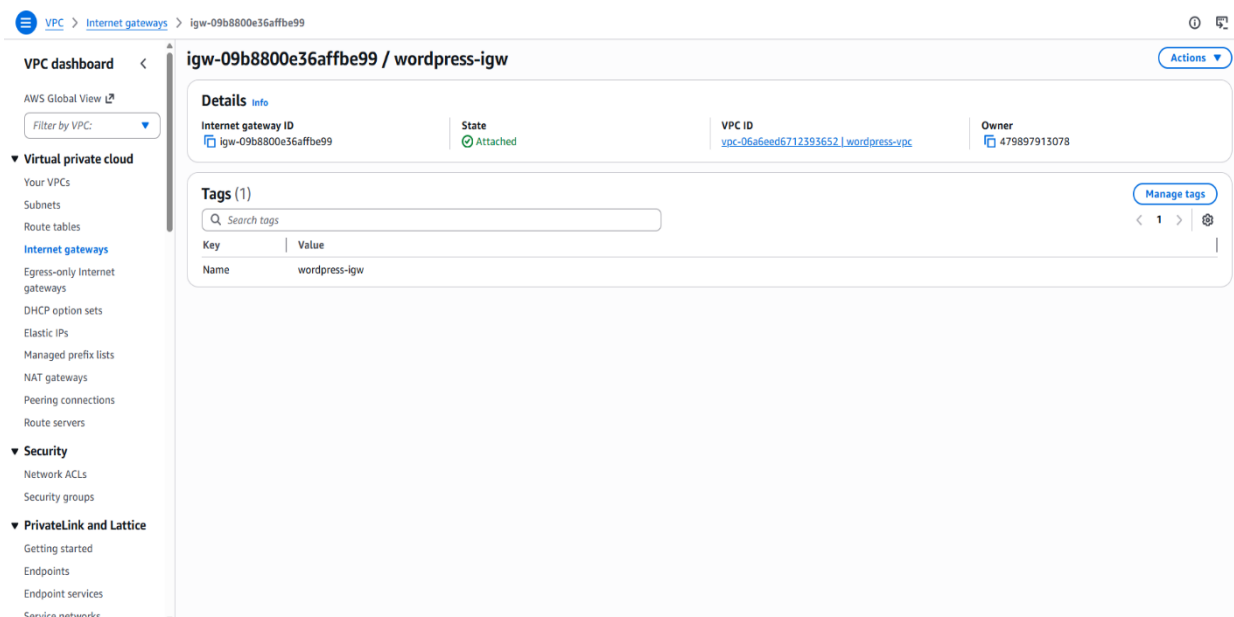
- Public Subnets

- Go to VPC → Subnets → Create Subnet.
- Select VPC: wordpress-vpc.
- Create Subnet 1:
 - Availability Zone: ap-south-1a
 - CIDR Block: 10.0.1.0/24
- Create Subnet 2:
 - Name: wordpress-public-subnet-2
 - Availability Zone: ap-south-1b
 - CIDR Block: 10.0.2.0/24
- Click Create Subnets.



STEP 3: Create Internet Gateway

- Name: wordpress-igw
- Click Create Internet Gateway
- Select IGW: Actions - Attach to VPC
- Click on Attach



STEP 4: Create Public Route Table

- Name: wordpress-rt
- VPC: wordpress-vpc
- Click create route table

Add Route:

- Select wordpress-rt
- Routes: Edit routes
- Add routes:
 - Destination: 0.0.0.0/0
 - Target: Internet Gateway (wordpress-igw)
 - Save routes

The screenshot shows the AWS VPC console interface for a specific route table. The left sidebar contains navigation links for VPC dashboard, Virtual private cloud, Security, and PrivateLink and Lattice. The main content area displays the details for route table 'rtb-0951f20ec06096b38 / wordpress-rt'. The 'Routes' tab is active, showing a table with two routes. The first route has destination '0.0.0.0/0', target 'igw-09b8800e36affbe99', and status 'Active'. The second route has destination '10.0.0.0/16', target 'local', and status 'Active'.

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-09b8800e36affbe99	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Step 5: Create Security Group

- Go to EC2 → Security Groups.
- Click Create Security Group.
- Name: wordpress-sg.
- Description: WordPress Security Group.
- VPC: wordpress-vpc.

Inbound Rules

- SSH (22) → My IP
- HTTP (80) → Anywhere (0.0.0.0/0)
- HTTPS (443) → Anywhere (0.0.0.0/0)
- MySQL/Aurora (3306) → wordpress-sg
- Click Create Security Group.

VPC > Security Groups > sg-0972ebd24ad512cc4 - wordpress-sg

VPC dashboard

AWS Global View

Filter by VPC:

▼ Virtual private cloud

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only Internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

▼ Security

- Network ACLs
- Security groups

▼ PrivateLink and Lattice

- Getting started
- Endpoints
- Endpoint services
- Service networks

sg-0972ebd24ad512cc4 - wordpress-sg

Details

Security group name wordpress-sg	Security group ID sg-0972ebd24ad512cc4	Description Security group for WordPress server	VPC ID vpc-06a6eed6712393652
Owner 479897913078	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (4)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-08e4d603899694cf5	IPv4	HTTP	TCP	80	0.0.0.0/0
-	sgr-043ebe7b37472457e	IPv4	SSH	TCP	22	103.252.26.117/32
-	sgr-0951c594f11a777b5	IPv4	HTTPS	TCP	443	0.0.0.0/0
-	sgr-025612c892bc1bac9	IPv4	All traffic	All	All	0.0.0.0/0

STEP 6: Created Subnet Group

- Go to RDS → Subnet Groups.
- Click Create DB Subnet Group.
- Name: wordpress-db-subnet-group.
- Description: WordPress Database Subnet Group.
- Select VPC: wordpress-vpc.
- Add both public subnets.
- Click Create.

Aurora and RDS > Subnet groups > wordpress-db-subnet-group

Aurora and RDS

- Dashboard
- Databases
- Performance insights
- Snapshots
- Exports in Amazon S3
- Automated backups
- Reserved instances
- Proxies
- Subnet groups
- Parameter groups
- Option groups
- Custom engine versions
- Zero-ETL integrations
- Events
- Event subscriptions
- Recommendations
- Certificate update

wordpress-db-subnet-group

Subnet group details

VPC ID
vpc-06a6eed6712393652

ARN
arn:aws:rds:ap-south-1:479897913078:subgrp:wordpress-db-subnet-group

Supported network types
IPv4

Description
Subnet group for WordPress RDS database

Subnets (2)

Availability zone	Subnet name	Subnet ID	CIDR block
ap-south-1b	wordpress-public-subnet-2	subnet-014b7d168988fcd1c	10.0.2.0/24
ap-south-1a	wordpress-public-subnet	subnet-0942d2386d8cace29	10.0.1.0/24

Tags (0)

Filter by Tag key

Tags

You don't have any tags associated with this resource.

STEP 7: Created Database

- Go to RDS → Create Database.
- Creation Method: Standard Create.
- Engine: MySQL.
- Template: Free Tier.
- DB Instance Identifier: wordpress-db.
- Master Username: admin.
- Set Database Password.
- Select VPC: wordpress-vpc.
- Select DB Subnet Group: wordpress-subnet-group.
- Security Group: wordpress-sg.
- Disable Public Access.
- Click Create Database.
- Wait until database status becomes Available.

The screenshot displays the AWS Management Console for the 'wordpressdb' RDS database instance. The left sidebar shows the navigation menu with 'Aurora and RDS' selected. The main content area shows the instance details under the 'Summary' tab. The instance is in the 'Available' state. The summary table includes the following information:

DB identifier	Status	Role	Engine	Recommendations
wordpressdb	Available	Instance	MySQL Community	
CPU	Class	Current activity	Region & AZ	
-	db.t4g.micro		ap-south-1a	

Below the summary, the 'Connectivity & security' tab is selected. It shows options to connect using 'Code snippets' (selected), 'CloudShell', or 'Endpoints'. The 'Internet access gateway' is disabled. The 'IAM Authentication' is also disabled. The 'Programming language' is set to 'MySQL (macOS)' and the 'Endpoint type' is 'Instance endpoint'. The 'Connection steps' section provides the following commands:

```
1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
1 mysql -h wordpressdb.cjaocm6kkdm.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem
```

STEP 8: Launch an instance

- Go to EC2 → Launch Instance.
- Name: wordpress-server.
- AMI: Amazon Linux 2023.
- Instance Type: t2.micro or t3.micro.
- Key Pair: aws-keypair.
- Network: wordpress-vpc.
- Subnet: wordpress-public-subnet-1.
- Auto Assign Public IP: Enable.

EC2

- Dashboard
- AWS Global View ↗
- Events
- ▼ **Instances**
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Elastic Reserved Instances
 - Capacity Manager [New](#)
- ▼ **Images**
 - AMIs
 - AMI Catalog
- ▼ **Elastic Block Store**
 - Volumes
 - Snapshots
 - Lifecycle Manager
- ▼ **Network & Security**
 - Security Groups

Breadcrumbs: EC2 > Instances > i-0ab19b4d696f791f6

Instance summary for i-0ab19b4d696f791f6 (wordpress-server) Info

Updated 15 minutes ago

Connect
Instance state ▼
Actions ▼

Instance ID i-0ab19b4d696f791f6	Public IPv4 address 15.206.145.50 open address	Private IPv4 addresses 10.0.1.213
IPv6 address -	Instance state Running	Public DNS ec2-15-206-145-50.ap-south-1.compute.amazonaws.com open address
Hostname type IP name: ip-10-0-1-213.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-1-213.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 15.206.145.50 [Public IP]	VPC ID vpc-06a6eed712393652 (wordpress-vpc)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0942d2386d8cace29 (wordpress-public-subnet)	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-south-1:479897913078:instance/i-0ab19b4d696f791f6	
Operator -		

[Details](#)
[Status and alarms](#)
[Monitoring](#)
[Security](#)
[Networking](#)
[Storage](#)
[Tags](#)


```

Installing      : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64      3/13
Installing      : apr-util-1.6.3-1.amzn2023.0.2.x86_64            4/13
Installing      : mailcap-2.1.49-3.amzn2023.0.3.noarch             5/13
Installing      : httpd-tools-2.4.67-1.amzn2023.0.1.x86_64        6/13
Installing      : libbrotli-1.0.9-4.amzn2023.0.2.x86_64           7/13
Running scriptlet: httpd-filessystem-2.4.67-1.amzn2023.0.1.noarch   8/13
Installing      : httpd-filessystem-2.4.67-1.amzn2023.0.1.noarch   8/13
Installing      : httpd-core-2.4.67-1.amzn2023.0.1.x86_64         9/13
Installing      : mod_http2-2.0.39-1.amzn2023.0.1.x86_64         10/13
Installing      : mod_lua-2.4.67-1.amzn2023.0.1.x86_64           11/13
Installing      : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 12/13
Installing      : httpd-2.4.67-1.amzn2023.0.1.x86_64            13/13
Running scriptlet: httpd-2.4.67-1.amzn2023.0.1.x86_64            13/13
Verifying      : apr-1.7.5-1.amzn2023.0.4.x86_64                1/13
Verifying      : apr-util-1.6.3-1.amzn2023.0.2.x86_64           2/13
Verifying      : apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64       3/13
Verifying      : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64    4/13
Verifying      : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 5/13
Verifying      : httpd-2.4.67-1.amzn2023.0.1.x86_64            6/13
Verifying      : httpd-core-2.4.67-1.amzn2023.0.1.x86_64       7/13
Verifying      : httpd-filessystem-2.4.67-1.amzn2023.0.1.noarch   8/13
Verifying      : httpd-tools-2.4.67-1.amzn2023.0.1.x86_64       9/13
Verifying      : libbrotli-1.0.9-4.amzn2023.0.2.x86_64         10/13
Verifying      : mailcap-2.1.49-3.amzn2023.0.3.noarch           11/13
Verifying      : mod_http2-2.0.39-1.amzn2023.0.1.x86_64        12/13
Verifying      : mod_lua-2.4.67-1.amzn2023.0.1.x86_64          13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-2.4.67-1.amzn2023.0.1.x86_64
httpd-core-2.4.67-1.amzn2023.0.1.x86_64
httpd-filessystem-2.4.67-1.amzn2023.0.1.noarch
httpd-tools-2.4.67-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_http2-2.0.39-1.amzn2023.0.1.x86_64
mod_lua-2.4.67-1.amzn2023.0.1.x86_64

```

i-0ab19b4d696f791f6 (wordpress-server)

PublicIPs: 15.206.145.50 PrivateIPs: 10.0.1.213

STEP 10: Start Apache and Download wordpress

- Start Apache:
- `sudo systemctl start httpd`
- Enable Apache:
- `sudo systemctl enable httpd`
- Verify Status:
- `sudo systemctl status httpd`

```

apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-2.4.67-1.amzn2023.0.1.x86_64
httpd-core-2.4.67-1.amzn2023.0.1.x86_64
httpd-filessystem-2.4.67-1.amzn2023.0.1.noarch
httpd-tools-2.4.67-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_http2-2.0.39-1.amzn2023.0.1.x86_64
mod_lua-2.4.67-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-1-213 ~]$ sudo systemctl start httpd
sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-1-213 ~]$ systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-06-01 15:05:42 UTC; 14s ago
     Docs: man:httpd.service(8)
  Main PID: 26125 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec 0; Bytes served/sec: 0"
     Tasks: 177 (limit: 1014)
    Memory: 13.5M
       CPU: 74ms
    CGroup: /system.slice/httpd.service
            └─26125 /usr/sbin/httpd -DFOREGROUND
              └─26126 /usr/sbin/httpd -DFOREGROUND
                └─26127 /usr/sbin/httpd -DFOREGROUND
                  └─26128 /usr/sbin/httpd -DFOREGROUND
                    └─26148 /usr/sbin/httpd -DFOREGROUND

Jun 01 15:05:42 ip-10-0-1-213.ap-south-1.compute.internal systemd[1]: Starting httpd.service:
Jun 01 15:05:42 ip-10-0-1-213.ap-south-1.compute.internal systemd[1]: Started httpd.service:
Jun 01 15:05:42 ip-10-0-1-213.ap-south-1.compute.internal httpd[26125]: Server configured, lis
[ec2-user@ip-10-0-1-213 ~]$

```

STEP 11 : We can see wordpress installation page Navigate to Web Directory:

`cd /var/www/html`

Download WordPress:

sudo wget <https://wordpress.org/latest.tar.gz>

Extract Files:

sudo tar -xvf latest.tar.gz

Copy Files:

sudo cp -r wordpress/* /var/www/html/

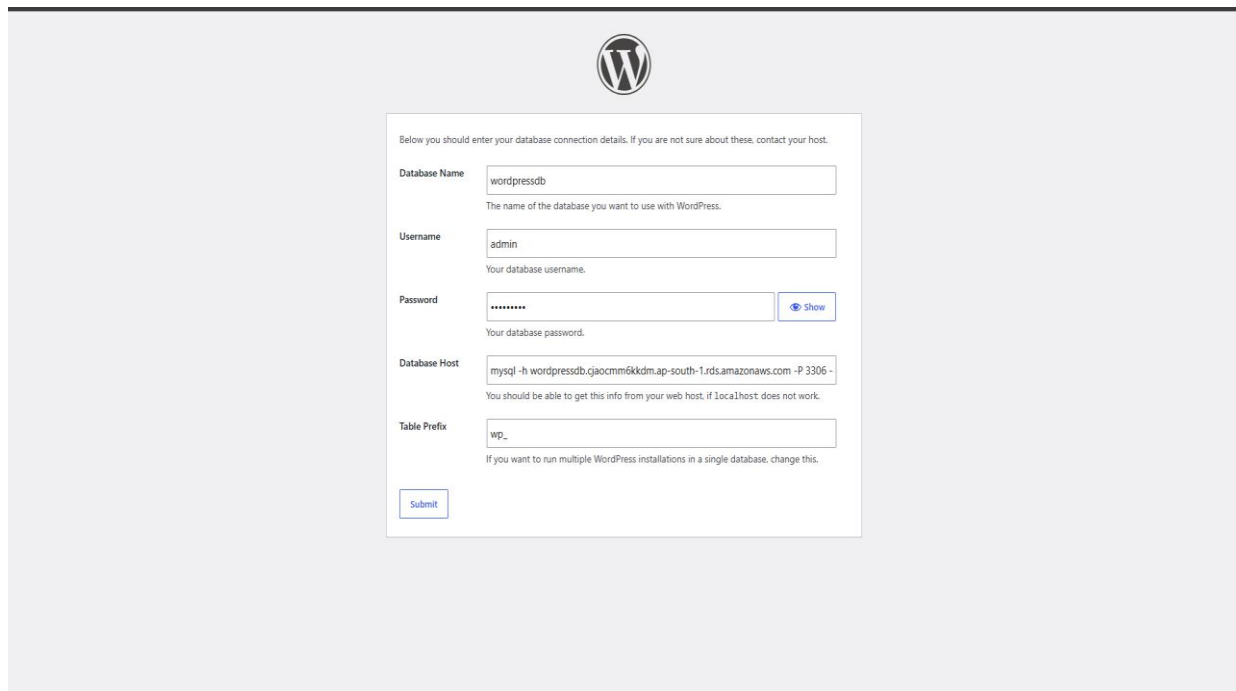
Set Permissions:

sudo chown -R apache /var/www/html

sudo chmod -R 755 /var/www/html

Restart Apache:

sudo systemctl restart httpd

The image shows the WordPress database connection screen. At the top is the WordPress logo. Below it, a text box says "Below you should enter your database connection details. If you are not sure about these, contact your host." The form has five input fields: "Database Name" with the value "wordpressdb", "Username" with the value "admin", "Password" with masked characters and a "Show" button, "Database Host" with the value "mysql -h wordpressdb.gjaocmm6kkdm.ap-south-1.rds.amazonaws.com -P 3306 -", and "Table Prefix" with the value "wp_". Each field has a small text description below it. At the bottom left is a "Submit" button.

STEP 12: We will connect wordpress → RDS database • Open Browser.

- Enter EC2 Public IP:

<http://Public-IP>

- Verify WordPress Setup Wizard appears.

- Select Language.

- Click Continue.

```
Server version: 8.0.46 Source distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| wordpressdb |
+-----+
5 rows in set (0.018 sec)

MySQL [(none)]> DROP DATABASE wordpressdb;
Query OK, 12 rows affected (0.200 sec)

MySQL [(none)]> CREATE DATABASE wordpressdb;
Query OK, 1 row affected (0.007 sec)

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| wordpressdb |
+-----+
5 rows in set (0.001 sec)

MySQL [(none)]> EXIT;
bye
[ec2-user@ip-10-0-1-213 html]$ sudo systemctl restart httpd
[ec2-user@ip-10-0-1-213 html]$
```

i-0ab19b4d696f791f6 (wordpress-server)

PublicIPs: 15.206.145.50 PrivateIPs: 10.0.1.213



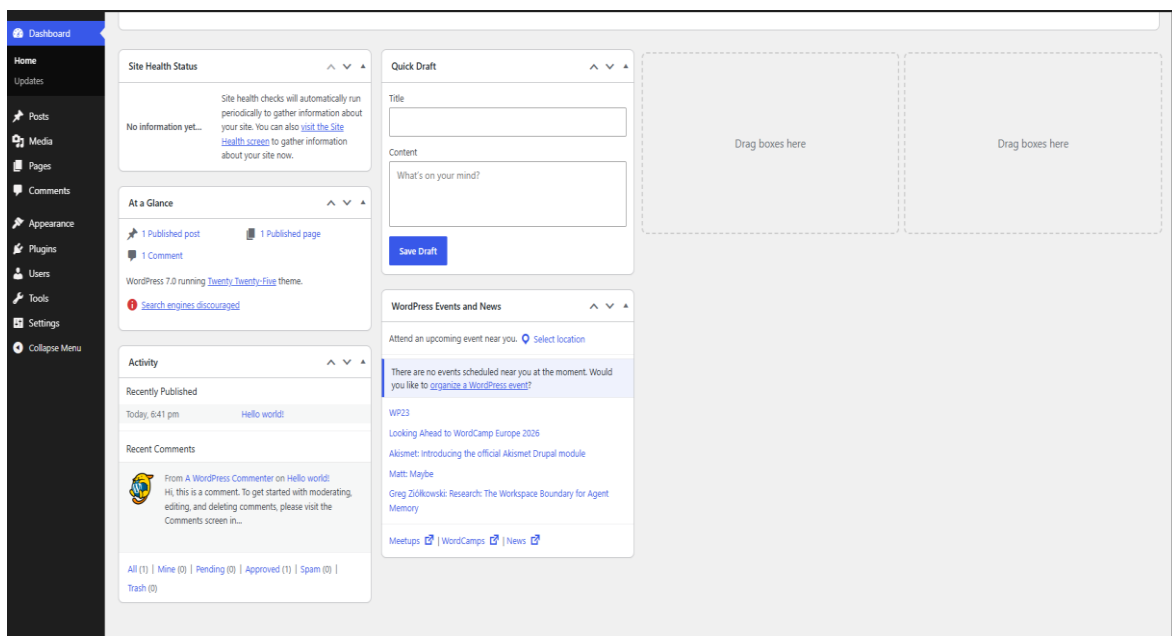
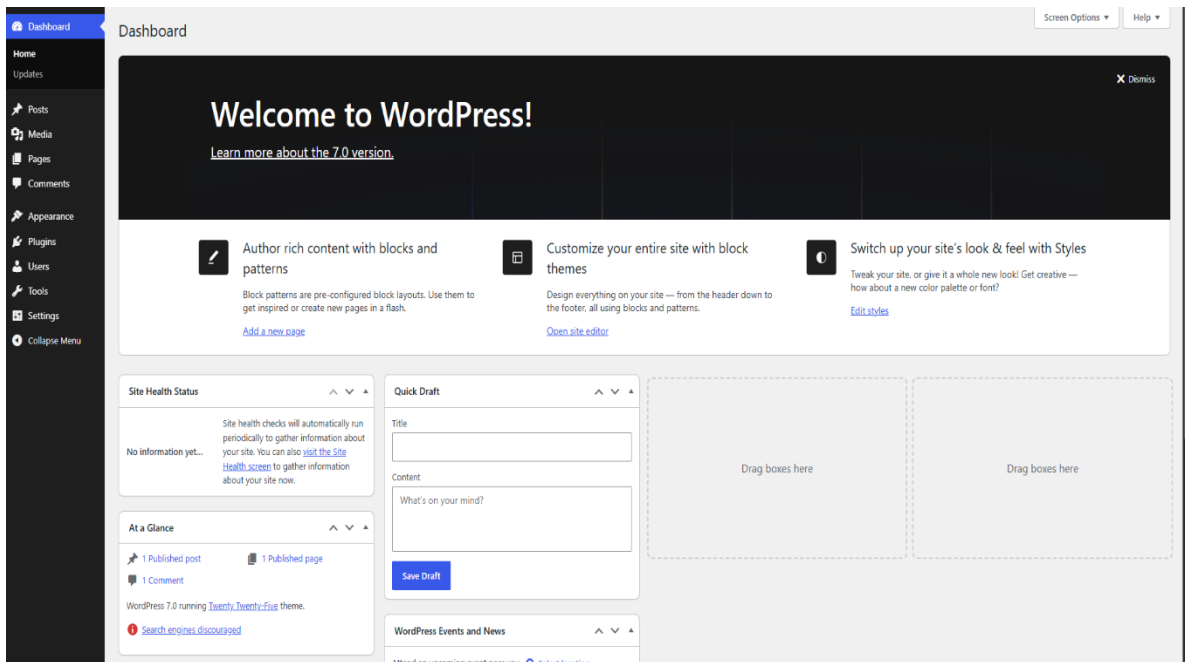
Success!

WordPress has been installed. Thank you, and enjoy!

Username maheshwari

Password Your chosen password.

[Log in](#)



Troubleshooting:

1. WordPress Database Connection Error

- Issue: WordPress displayed “Error establishing database connection” during setup.
- Solution: Updated the wp-config.php file with the correct database name, username, password, and RDS endpoint.

2. EC2 Unable to Connect to RDS

- Issue: The EC2 instance could not connect to the MySQL database on Amazon RDS.
- Solution: Added a MySQL inbound rule (Port 3306) in the RDS security group and allowed access from the EC2 security group.

Conclusion:

This project successfully demonstrated the deployment of a highly available WordPress website on AWS using services such as Amazon EC2, Amazon RDS, EFS, ALB and CloudFront. The implementation provided hands-on experience in cloud infrastructure, database management, networking, and security. It also showed how AWS services can be integrated to create a scalable, reliable, and efficient web hosting environment. Overall, the project highlights the benefits of cloud-native architecture for deploying and managing modern web applications.